

Universidade Tecnológica Federal do Paraná
Engenharia da Computação

RPG Manager

Sistema de Gerenciamento de Jogo de RPG

Especificação Técnica e Arquitetura do Sistema

Disciplina: Programação Orientada a Objetos

Carga Horária: 40 horas (Projeto Semestral)

Apucarana, 2026

Sumário

1	Introdução e Motivação	2
2	Arquitetura de Classes e Visibilidade	2
2.1	Módulo de Entidades e Personagens	2
2.1.1	Classe Abstrata: Entidade	2
2.1.2	Classe: Personagem (Herda de Entidade)	3
2.1.3	Classes Concretas: Especializações de Classe	3
2.2	Módulo de Inventário e Itens	4
2.2.1	Classe Abstrata: Item	4
2.2.2	Classe: Inventario	4
3	Requisitos do Sistema	4
3.1	Requisitos Funcionais	4
3.2	Requisitos Técnicos	5
4	Métricas de Avaliação (UTFPR)	5

1 Introdução e Motivação

Jogos de RPG (*Role-Playing Game*) são sistemas complexos que envolvem múltiplas entidades com atributos, comportamentos e relacionamentos bem definidos, tornando-os um domínio extremamente rico para a prática de Programação Orientada a Objetos (POO) [?]. A modelagem de personagens, habilidades, inventários e regras de combate é um exercício natural de abstração, encapsulamento, herança e polimorfismo [?].

Este documento especifica o desenvolvimento do *RPG Manager*, um sistema de gerenciamento de jogo de RPG implementado em C++17 rodando via interface de linha de comando (CLI) [?, ?, ?].

2 Arquitetura de Classes e Visibilidade

Para garantir as boas práticas de design de software, os modificadores de acesso foram rigidamente definidos, utilizando o encapsulamento e a herança protegida onde conveniente.

2.1 Módulo de Entidades e Personagens

2.1.1 Classe Abstrata: Entidade

Base para todos os seres vivos e NPCs do jogo [?].

- **Atributos Protegidos (protected):**

- `std::string nome;`
- `int pontos_vida_max;`
- `int pontos_vida_atual;`
- `int nivel;`

- **Métodos Públicos (public):**

- Construtor e Destrutor virtual padrão.
- `virtual void receber_dano(int dano) = 0;` (*Função virtual pura*).
- Getters básicos para as propriedades vitais.

2.1.2 Classe: Personagem (Herda de Entidade)

Adiciona os elementos customizáveis da ficha do jogador [?].

- **Atributos Privados (private):**

- `std::string raca;`
- `std::string classe_rpg;`
- `int experiencia;`
- `Inventario inventario;` (*Composição*) [?, ?].
- `std::vector<std::unique_ptr<Habilidade>> habilidades;` (*Agregação Polimórfica*) [?].

- **Métodos Públicos (public):**

- `void equipar_item(std::shared_ptr < Item > item);` [?].
- `void desequipar_item(std::string nome_item);` [?].
- `void usar_item(std::string nome_item);` [?].
- `void ganhar_experiencia(int exp_ganha);` (*Gatilha o Level Up*) [?].
- `void atacar(Entidade& alvo, int index_habilidade);`
- `void exibir_status() const;` [?].

2.1.3 Classes Concretas: Especializações de Classe

Classes como Guerreiro, Mago, Arqueiro e Clerigo estendem os comportamentos básicos [?].

- **Mago:** Adiciona o atributo privado `int mana_max` e `int mana_atual`.
- **Guerreiro:** Adiciona o recurso privado `int furia` ou `estamina`.
- **Método Especializado:** Sobrescrita (*override*) do método `receber_dano(int dano)` para aplicar mitigações de armadura ou escudos mágicos específicos de cada classe.

—

2.2 Módulo de Inventário e Itens

2.2.1 Classe Abstrata: Item

Molde polimórfico para qualquer objeto coletável [?].

- **Atributos Protegidos (protected):**

- `std::string nome;`
 - `double peso;`
 - `int valor_moedas;`

- **Métodos Públicos (public):**

- `virtual void usar(Personagem& usuario) = 0;`

2.2.2 Classe: Inventario

- **Atributos Privados (private):**

- `std::vector<std::shared_ptr < Item >> itens; [?].`
 - `double capacidade_peso_max; [?].`
 - `double peso_atual; [?].`

- **Métodos Públicos (public):**

- `bool adicionar_item(std::shared_ptr < Item > item);` (*Verifica regras de pes*
 - `void remover_item(std::string nome_item); [?].`
 - `void listar_itens() const;`

3 Requisitos do Sistema

3.1 Requisitos Funcionais

1. Criação de personagens parametrizada (nome, raça, classe e atributos) [?].
2. Listagem e detalhamento da ficha dos heróis criados [?].

3. Sistema de gerenciamento e uso de itens do inventário [?].
4. Equipar/Desequipar armas e armaduras com recálculo automático de atributos [?].
5. Combate em turnos: suporte para Jogador vs Jogador e Jogador vs Bot [?].
6. Sistema de Experiência e *Level Up* com alocação manual de pontos [?].
7. Persistência do estado do jogo (Salvar/Carregar) em arquivos texto ou JSON [?].
8. Interface amigável via Linha de Comando (CLI) [?].

3.2 Requisitos Técnicos

- Uso extensivo de **Ponteiros Inteligentes** (`std::unique_ptr` e `std::shared_ptr`) para evitar...
- Sobrecarga de Operadores: Implementação semântica dos operadores `<`, `==` e `+` para ordenar ou comparar entidades [?].
- Organização modular estruturada em namespaces e arquivos `.h` e `.cpp` distintos [?].
- Uso da STL (`vector`, `map`, `algorithm`) [?].
- Cobertura mínima de 60% do código crítico através de Testes Unitários (Google Test) [?, ?].
- Código documentado seguindo os padrões do Doxygen [?].
- Pelo menos 4 Padrões de Projeto aplicados (Ex: *Strategy* para IA, *Observer* para logs de batalha, *Factory* para itens e *Singleton* para a persistência) [?].

4 Métricas de Avaliação (UTFPR)

O projeto será avaliado com base na composição de pesos descrita na Tabela 1 [?, ?].

Tabela 1: Distribuição de Notas do Projeto

Critério de Avaliação	Peso (%)	Pontuação Máxima
Implementação correta dos conceitos de POO	30%	30 pts
Funcionamento correto do sistema (Requisitos)	25%	25 pts
Qualidade do código e Boas Práticas	15%	15 pts
Diagrama de Classes UML	15%	15 pts
Apresentação oral e Relatório Técnico	15%	15 pts